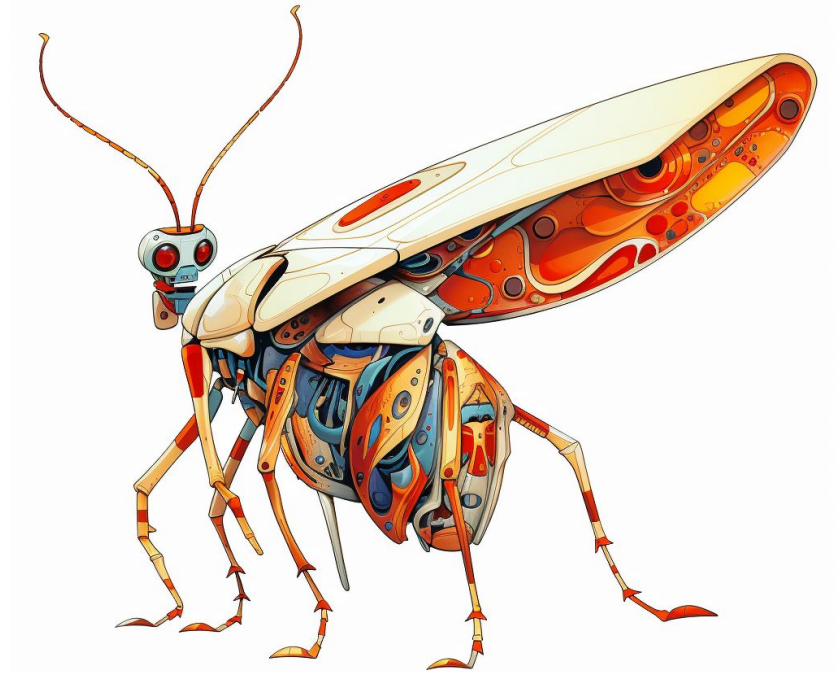


Debugging

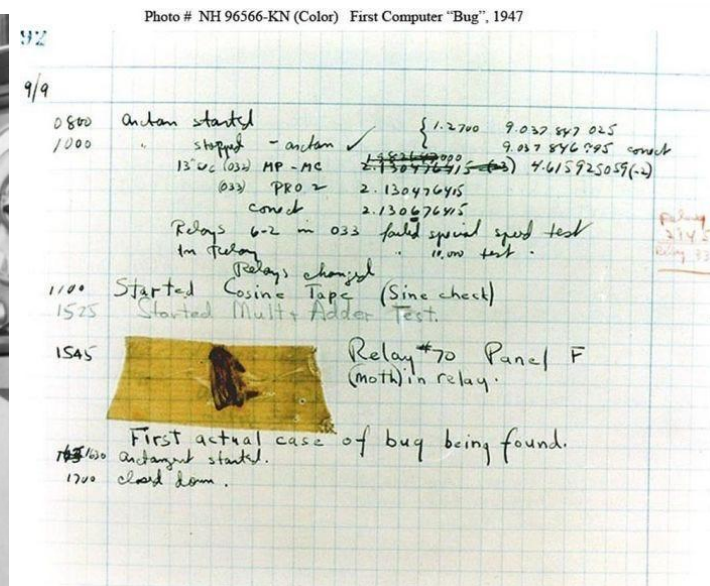
How to fix problems

Lisa, Carlos, and Ece



The very first computer bug

Team around **Grace Hopper** at Harvard found a moth in their computer in 1947 in what is maybe the first description of a computer bug



Debugging and the agile development cycle

- By writing tests, you *anticipate* the bugs
- Your test cases *should already* exclude a big portion of the possible causes

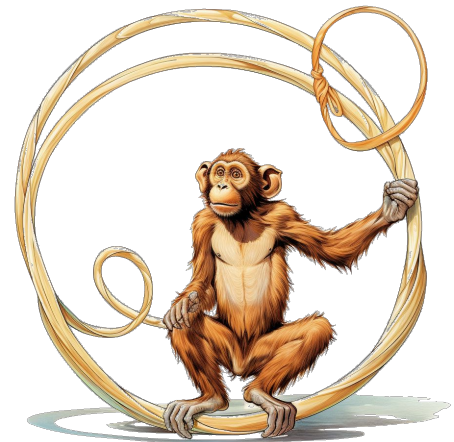
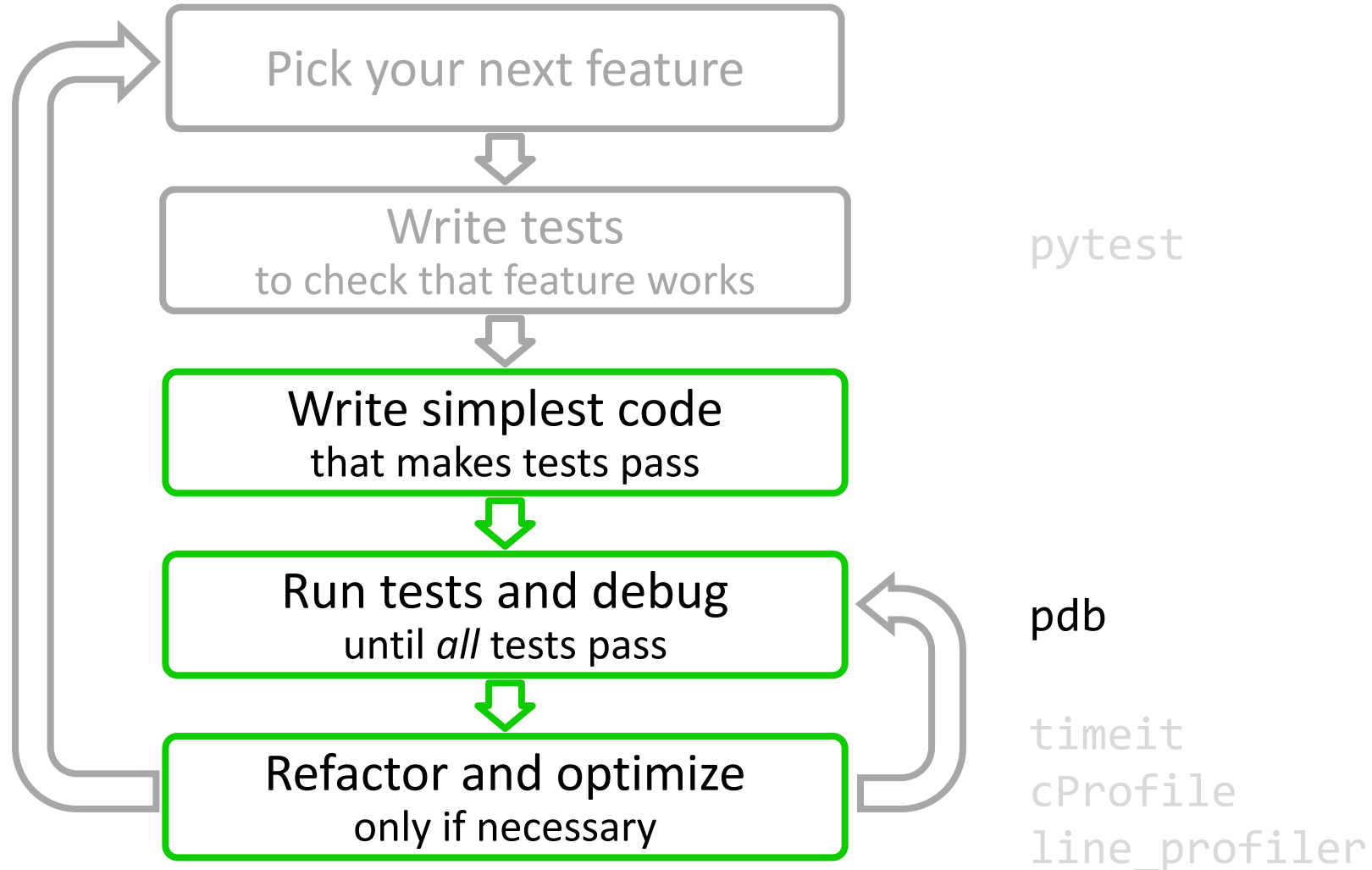


Debugging and the agile development cycle

- By writing tests, you *anticipate* the bugs
- Your test cases *should already* exclude a big portion of the possible causes



Debugging and the agile development cycle



Debugging and the agile development cycle

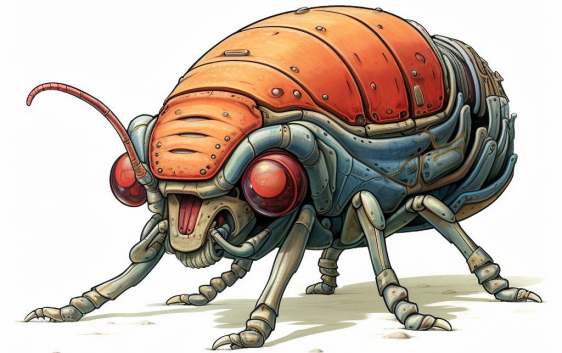
1. Add a test that matches the behavior you expect. It will fail and reproduce the bug
2. *Debug and fix the the bug*
3. Run the tests until they all pass (go back to 2 if necessary)

Now your bug is fixed *and* it will never occur again! (:



pdb, the Python debugger

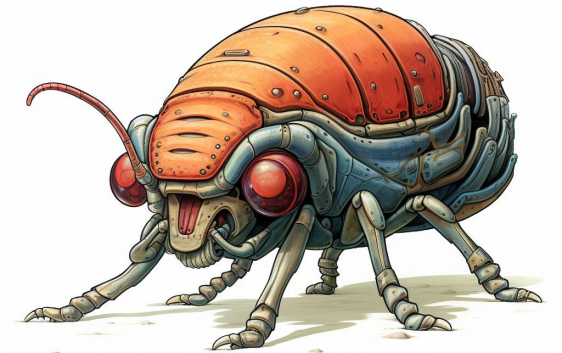
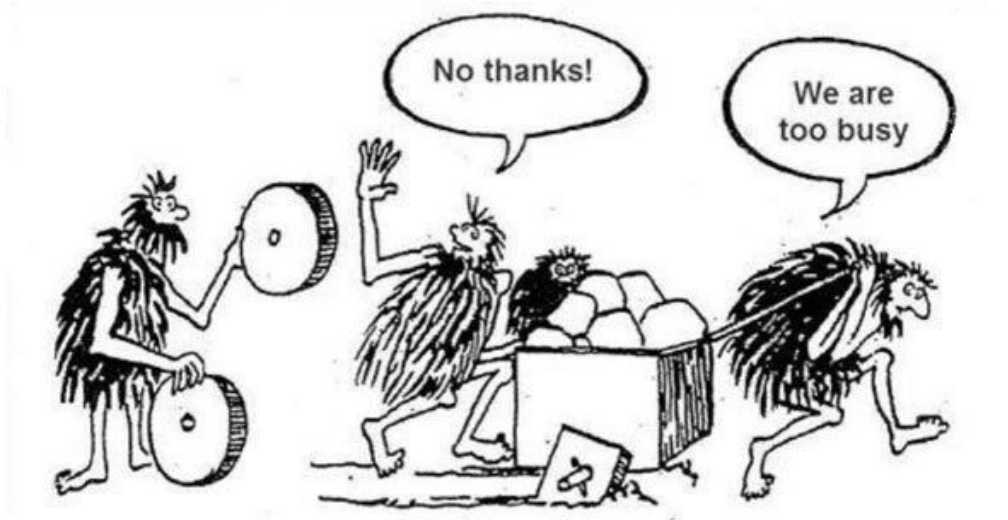
- Core idea in debugging: you can stop the execution of your application at any point, look at the state of the variables, and execute the code step by step
- Command-line based debugger
- pdb opens an interactive shell, in which one can interact with the code
 - examine and change value of variables
 - execute code line by line
 - set up breakpoints
 - examine calls stack



pdb, the Python debugger

“But I have

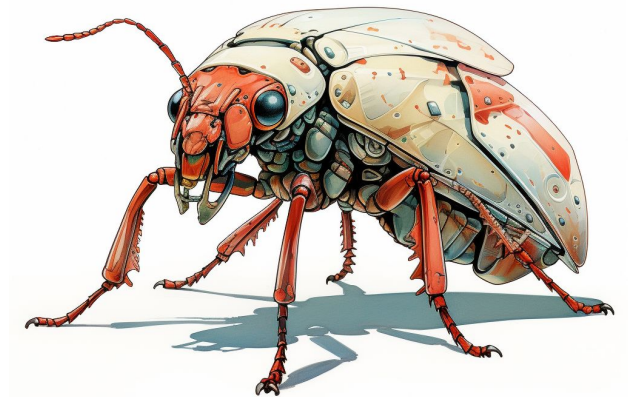
- print statements that I added*
- more print statements that I can add*
- I can copy&paste portions of code individually*
- friends & colleagues?*
- AI??!...? 🙄”*



Entering the debugger

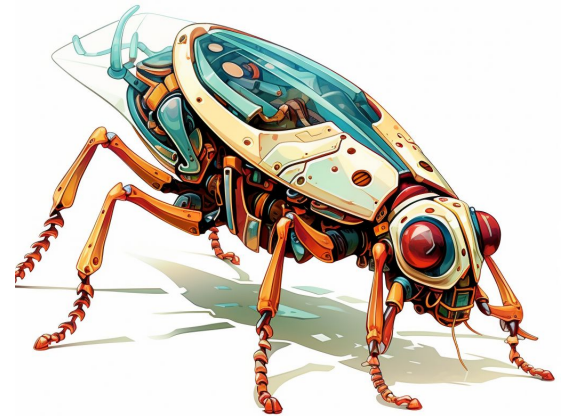
- Enter at a specific point in the code (easy alternative to print):

```
# some code here  
# the debugger starts here  
breakpoint()  
# rest of the code
```



Demonstration!

- Debugging with pdb



Hands-on! – Issue #2

1) Go to `local_maxima_part3_debug/local_maxima.py` and execute it

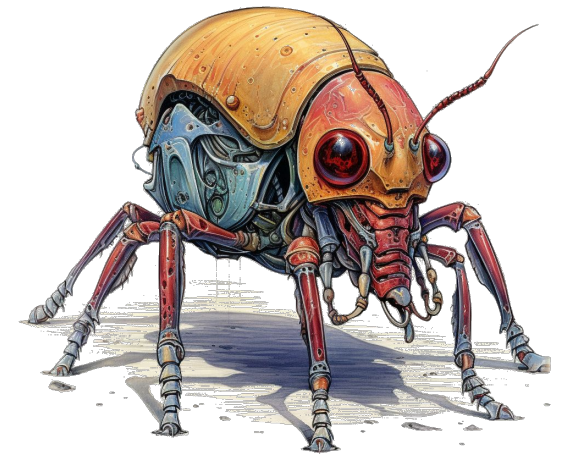
2) Run the script with these inputs:

- `[1]` (expected output: `[0]`),
- `[2, 2, 1, 0]` (expected output: `[0]`)

3) When you find a bug, write a test, then debug and fix it. Finally, create a PR

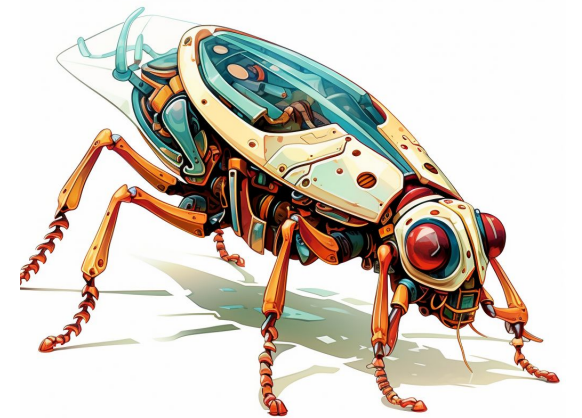
pdb cheatsheet:

<code>h (help) [command]</code>	print help about <i>command</i>
<code>n (next)</code>	execute current line of code, go to next line
<code>c (continue)</code>	continue executing the program until next breakpoint, exception, or end of the program
<code>s (step into)</code>	execute current line of code; if a function is called, follow execution inside the function
<code>l (list)</code>	print code around the current line
<code>w (where)</code>	show a trace of the function call that led to the current line
<code>p (print)</code>	print the value of a variable
<code>q (quit)</code>	leave the debugger
<code>b (break) [lineno function[, condition]]</code>	set a breakpoint at a given line number or function, stop execution there if <i>condition</i> is fulfilled
<code>cl (clear)</code>	clear a breakpoint
<code>! (execute)</code>	execute a python command
<code><enter></code>	repeat last command

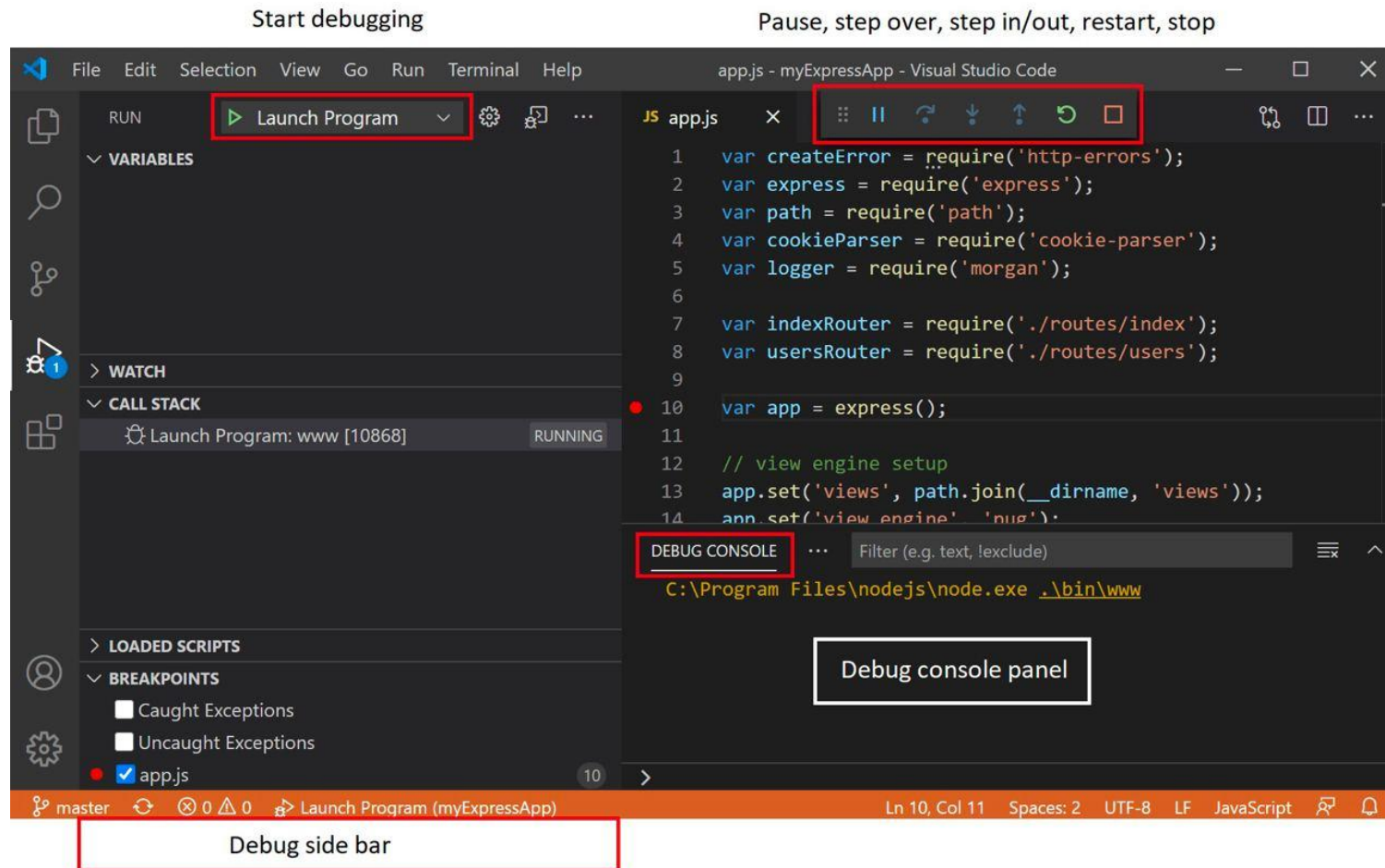


Demonstration!

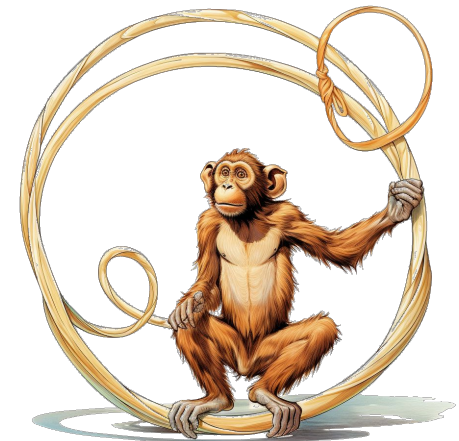
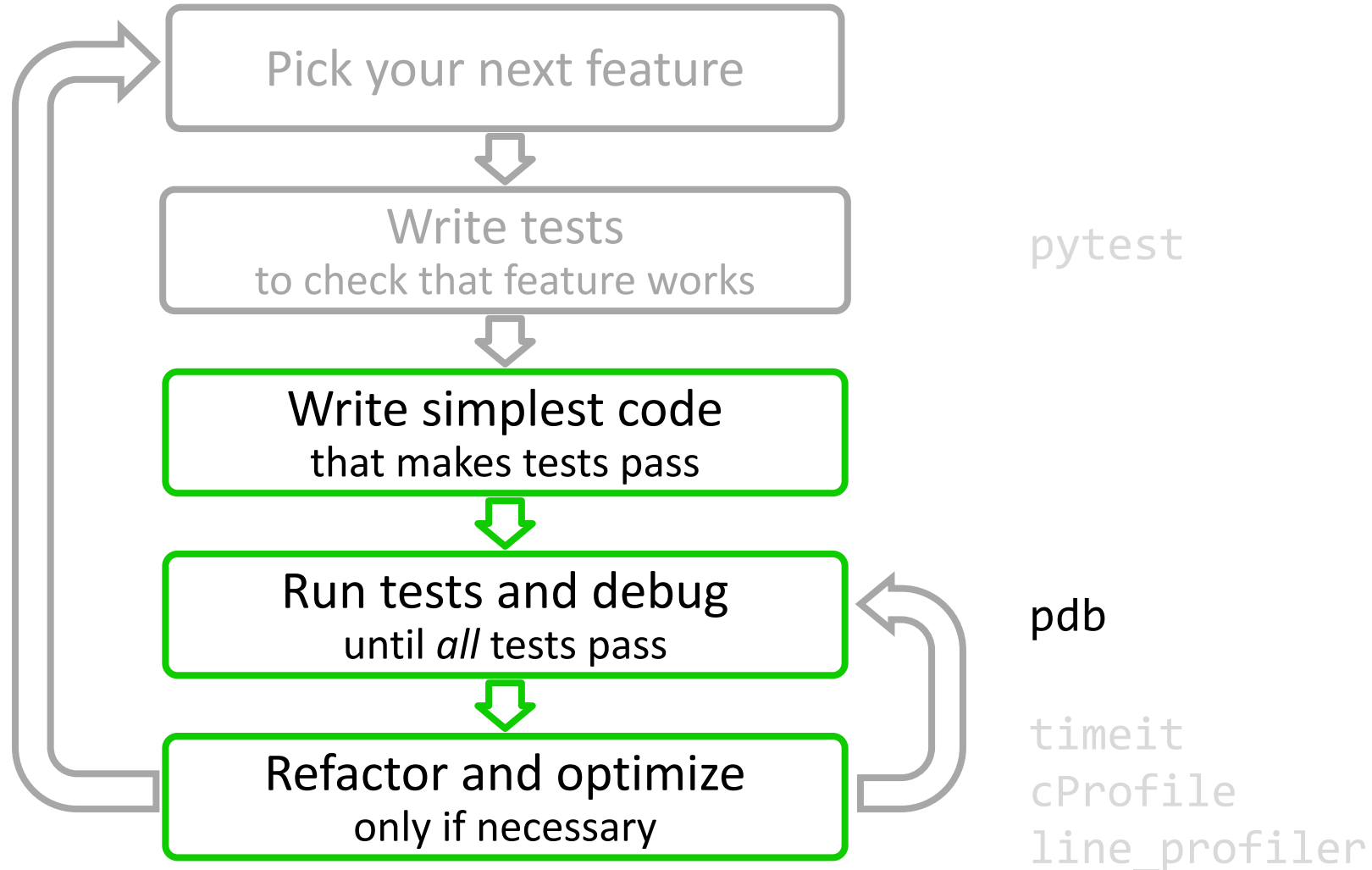
- Debugging with pdb
 - ... *in ipython*
- Debugging from an IDE



Entering the debugger from VSCode

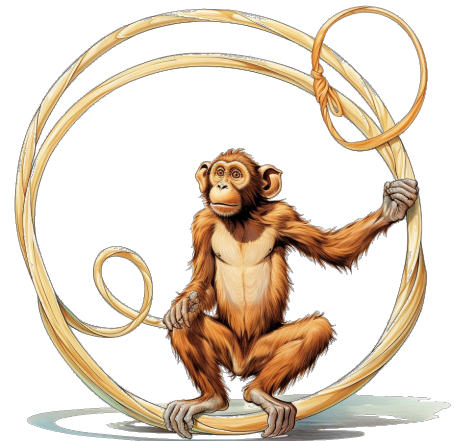
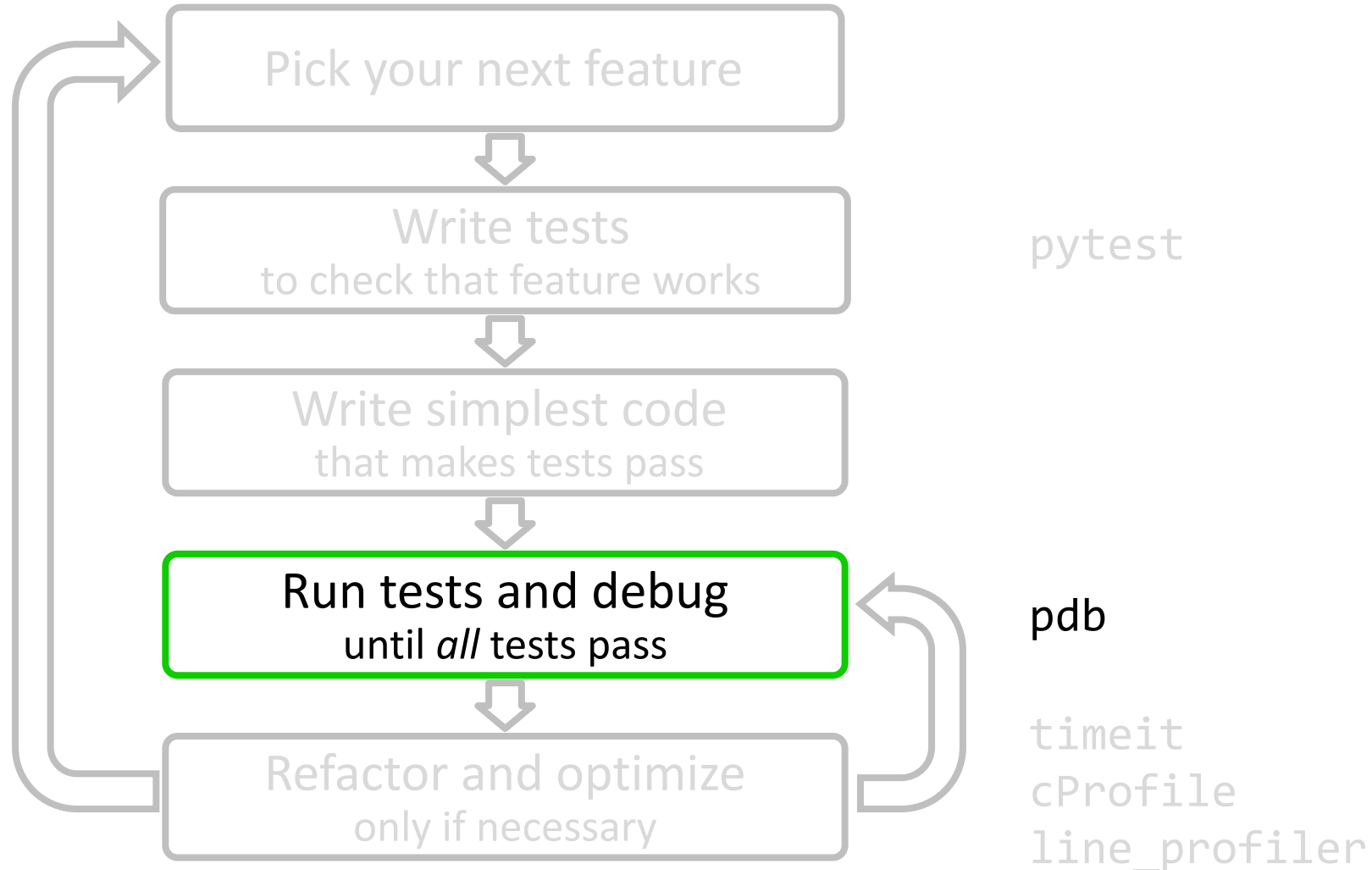


Test, debug, then commit and create a PR



The End

The agile development cycle



talk about testing directory and that it only works to have tests in a different folder if you have installed your code as a package

Entering the debugger from Jupyter

- `%pdb` – preventive
- `%debug` – post-mortem



Hands-on!

- Go to `bug_hunt/file_datastore.py` and execute it

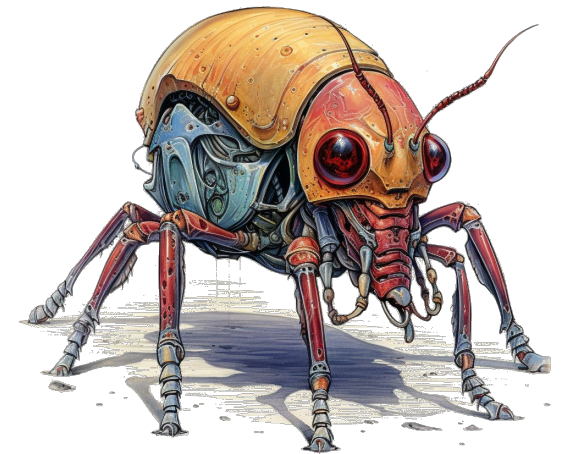
```
data = b'A test! 012'
datastore = FileDatastore(base_path='./datastore')
datastore.write('a/mydata.bin', data)

# This should pass!
assert os.path.exists('./datastore/a/mydata.bin')
```

- It fails! But... it works when the `base_path` is an absolute path :-)

Change this exercise!!!!

Maybe debug a script or
maybe even better - a
test?



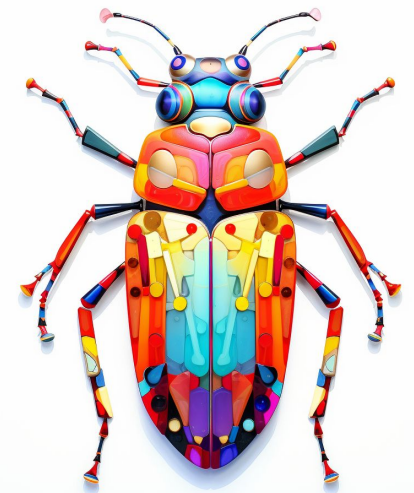
Static checking and linting

One of the problems with debugging in Python is that most errors only appear when the code executes.

“Static checking” tools analyze the code without executing it.

- pep8: check that the style of the files is compatible with PEP 8
- pyflakes: look for errors like defined but unused variables, syntax errors, etc.
- flake8: pep8 and pyflakes in a single, handy command
- and also: yapf, black, ...

Modify this and adapt to not have overlap with organising lecture

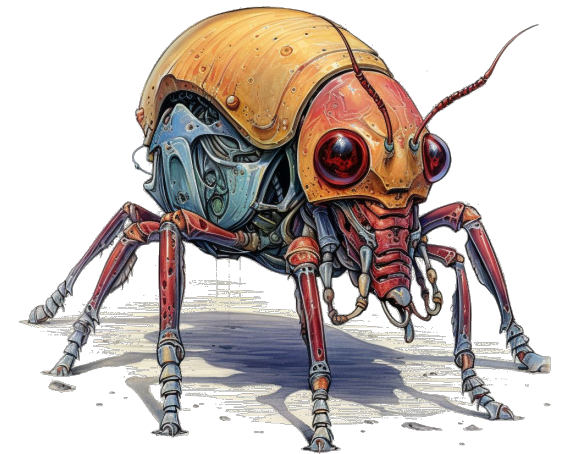


Hands-on!

- Fix the bug in `file_datastore.py`, using the debugger
- Submit a PR for issue #1 in the repository

pdb cheatsheet

<code>h (help) [<i>command</i>]</code>	print help about <i>command</i>
<code>n (next)</code>	execute current line of code, go to next line
<code>c (continue)</code>	continue executing the program until next breakpoint, exception, or end of the program
<code>s (step into)</code>	execute current line of code; if a function is called, follow execution inside the function
<code>l (list)</code>	print code around the current line
<code>w (where)</code>	show a trace of the function call that led to the current line
<code>p (print)</code>	print the value of a variable
<code>q (quit)</code>	leave the debugger
<code>b (break) [<i>lineno</i> <i>function</i>[, <i>condition</i>]]</code>	set a breakpoint at a given line number or function, stop execution there if <i>condition</i> is fulfilled
<code>cl (clear)</code>	clear a breakpoint
<code>! (execute)</code>	execute a python command
<code><enter></code>	repeat last command



Tools for Debugging

- Text editor + terminal □ pdb (Python's built-in debugger)
- VSCode (IDE) □ VSCode built-in debugger
- PyCharm (IDE) □ PyCharm built-in debugger
- Jupyter □ (?) built-in debugger



Hands-on!

- Run `flake8` on one the files you edited today

